

Tutorial – Strings na Linguagem C

1. Introdução

Na linguagem C, uma string é representada por um vetor de caracteres (**char**).

Exemplo:

```
C/C++  
char nome[7];
```

Cada posição do vetor armazena um caractere individual.

2. O Tipo **char**

O tipo **char** é utilizado para armazenar caracteres da tabela ASCII.

Exemplo:

```
C/C++  
char letra = 'A';
```

Cada caractere ocupa normalmente 1 byte de memória.

3. Criando Strings

3.1 Inicialização manual

Uma string pode ser criada caractere por caractere:

```
C/C++  
char nome[7] = {'F', 'l', 'a', 'v', 'i', 'o'};
```

Entretanto, essa abordagem possui um problema importante: falta o caractere terminador da string.

4. Terminador ' \0 '

Em C, toda string deve terminar com o caractere especial:

```
C/C++  
' \0 '
```

Esse caractere indica o fim da string.

A representação correta seria:

```
C/C++  
char nome[7] = {'F', 'l', 'a', 'v', 'i', 'o', '\0'};
```

5. String Literal

A forma mais prática de criar strings é utilizando aspas duplas:

```
C/C++  
char nome[7] = "Flavio";
```

O compilador automaticamente adiciona o ' \0 '.

6. Imprimindo Strings

Para imprimir strings utilizamos %s no printf():

```
C/C++  
#include <stdio.h>  
  
int main() {  
  
    char nome[] = "Flavio";
```

```
    printf("%s\n", nome);

    return 0;
}
```

7. Diferença entre `strlen()` e `sizeof()`

Exemplo

```
C/C++
#include <stdio.h>
#include <string.h>

int main() {

    char nome[] = "Carlos";

    printf("strlen: %d\n", strlen(nome));
    printf("sizeof: %d\n", sizeof(nome));

    return 0;
}
```

Saída

```
None
strlen: 6
sizeof: 7
```

Explicação

- `strlen()` conta apenas os caracteres visíveis.
- `sizeof()` conta também o `'\0'`.

8. Biblioteca `string.h`

A biblioteca `string.h` fornece funções prontas para manipulação de strings.

Para utilizá-la:

```
C/C++  
#include <string.h>
```

9. Função `strcpy()`

A função `strcpy()` copia uma string para outra.

Sintaxe

```
C/C++  
strcpy(destino, origem);
```

Exemplo

```
C/C++  
#include <stdio.h>  
#include <string.h>  
  
int main() {  
  
    char origem[] = "IFCE";  
    char destino[20];  
  
    strcpy(destino, origem);  
  
    printf("%s\n", destino);  
  
    return 0;  
}
```

Saída

None
IFCE

10. Função `strcat()`

A função `strcat()` concatena strings.

Sintaxe

C/C++
`strcat(destino, origem);`

Exemplo

```
C/C++
#include <stdio.h>
#include <string.h>

int main() {

    char nome[30] = "Ciencia ";
    char curso[] = "Computacao";

    strcat(nome, curso);

    printf("%s\n", nome);

    return 0;
}
```

Saída

None
Ciencia Computacao

11. Função `strcmp()`

A função `strcmp()` compara duas strings.

Sintaxe

C/C++

```
strcmp(str1, str2);
```

Retorno

Resultado	Significado
0	Strings iguais
< 0	Primeira string menor
> 0	Primeira string maior

Exemplo

C/C++

```
#include <stdio.h>
#include <string.h>

int main() {

    char s1[] = "abc";
    char s2[] = "abc";

    if(strcmp(s1, s2) == 0) {
        printf("Strings iguais\n");
    }

    return 0;
}
```

12. Função `strncmp()`

Compara apenas os primeiros `n` caracteres.

Exemplo

```
C/C++
#include <stdio.h>
#include <string.h>

int main() {

    char s1[] = "computador";
    char s2[] = "computacao";

    if(strncmp(s1, s2, 6) == 0) {
        printf("Os 6 primeiros caracteres sao iguais\n");
    }

    return 0;
}
```

13. Função `strlen()`

Calcula o tamanho da string.

Exemplo

```
C/C++
#include <stdio.h>
#include <string.h>

int main() {

    char palavra[] = "Maratona";

    printf("Tamanho: %d\n", strlen(palavra));
}
```

```
    return 0;
}
```

14. Função **strchr()**

Procura a primeira ocorrência de um caractere.

Exemplo

```
C/C++
#include <stdio.h>
#include <string.h>

int main() {

    char texto[] = "programacao";

    char *resultado = strchr(texto, 'g');

    if(resultado != NULL) {
        printf("Encontrado: %c\n", *resultado);
    }

    return 0;
}
```

15. Função **strstr()**

Procura uma substring dentro de outra string.

Exemplo

```
C/C++
#include <stdio.h>
```



```
#include <string.h>

int main() {

    char texto[] = "linguagem c";

    if(strstr(texto, "c") != NULL) {
        printf("Substring encontrada\n");
    }

    return 0;
}
```

16. Função **memset()**

Muito utilizada para limpar strings ou vetores.

Exemplo

```
C/C++
#include <stdio.h>
#include <string.h>

int main() {

    char texto[20] = "IFCE";

    memset(texto, '\0', sizeof(texto));

    printf("%s\n", texto);

    return 0;
}
```

17. Leitura de Strings

17.1 Usando `scanf("%s")`

```
C/C++
char nome[20];

scanf("%s", nome);
```

Problema:

- A leitura para no primeiro espaço.

Exemplo

Entrada:

```
None
Ana Clara
```

Saída:

```
None
Ana
```

17.2 Usando `scanf(" %[^\n]")`

Para permitir leitura com espaços:

```
C/C++
char nome[100];

scanf(" %[^\n]", nome);
```

Exemplo Completo

```
C/C++
#include <stdio.h>

int main() {
```

```
char nome[100];

scanf(" %[^\\n]", nome);

printf("%s\\n", nome);

return 0;
}
```

Entrada

```
None
Instituto Federal do Ceara
```

Saída

```
None
Instituto Federal do Ceara
```

18. Leitura com **fgets()**

A função **fgets()** é uma das formas mais seguras de ler strings.

Exemplo

```
C/C++
#include <stdio.h>

int main() {

    char nome[50];

    fgets(nome, 50, stdin);

    printf("%s", nome);
}
```

```
    return 0;
}
```

19. Removendo o `\n` do `fgets()`

O `fgets()` normalmente armazena o ENTER (`\n`).

Para removê-lo:

```
C/C++
nome[strcspn(nome, "\n")] = '\0';
```

Exemplo Completo

```
C/C++
#include <stdio.h>
#include <string.h>

int main() {

    char nome[50];

    fgets(nome, 50, stdin);

    nome[strcspn(nome, "\n")] = '\0';

    printf("%s\n", nome);

    return 0;
}
```

20. Cuidados Importantes

Nunca usar `gets()`

A função `gets()` é insegura e foi removida do padrão C.

Garantir espaço suficiente

Sempre reserve espaço para o `'\0'`.

Exemplo:

C/C++

```
char nome[6] = "Pedro";
```

21. Resumo das Principais Funções

Função	Objetivo
<code>strlen()</code>	calcular tamanho
<code>strcpy()</code>	copiar strings
<code>strcat()</code>	concatenar strings
<code>strcmp()</code>	comparar strings
<code>strncmp())</code>	comparar parte das strings
<code>strchr()</code>	procurar caractere
<code>strstr()</code>	procurar substring
<code>memset()</code>	limpar memória/string

22. Conclusão

A manipulação de strings em C exige atenção porque:

- Strings são vetores de caracteres;
- O programador controla diretamente a memória;

- É necessário cuidar do terminador '`\0`'.

A biblioteca `string.h` simplifica bastante o trabalho oferecendo funções prontas para:

- copiar;
- comparar;
- concatenar;
- medir;
- procurar strings.

Atividades:

Exercício 1 – Fácil

Desenvolva um programa em C que leia o nome de uma pessoa e exiba: - quantidade de caracteres do nome; - primeira letra; - última letra. Utilize a função `strlen()`.

Exercício 2 – Médio

Desenvolva um programa que leia duas strings e informe: - se elas são iguais; - qual delas é maior em ordem alfabética. Utilize `strcmp()`.

Exercício 3 – Médio

Desenvolva um programa que leia nome e sobrenome separadamente, concatene as duas strings e exiba o nome completo. Utilize `strcat()`.

Exercício 4 – Difícil

Desenvolva um programa que leia uma frase e conte: - quantidade de vogais; - quantidade de consoantes; - quantidade de espaços.

Exercício 5 – Difícil

Desenvolva um programa que verifique se uma palavra é um palíndromo. Exemplo: ANA -> palíndromo CASA -> não é palíndromo A verificação deve ser feita manualmente utilizando vetores e laços de repetição.